



Full Transformer Architecture

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

সম্পূর্ণ Transformer Architecture — বিস্তারিত বাংলা নোট

Topic: The Full Transformer Architecture — Encoder Block, Decoder Block, Add & Norm, Feed Forward Networks

Series: Transformers & LLMs — পর্ব ৫ (শেষ পর্ব)

পূর্বশর্ত: Self-Attention (Q, K, V), Multi-Head Attention, Positional Encoding

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

Transformer কী এবং কেন দরকার?

কল্পনা করো তুমি একটি বিশাল অনুবাদ সংস্থার CEO। তোমার কাছে ইংরেজি থেকে বাংলায় অনুবাদ করার কাজ আসে।

তোমার সংস্থায় দুটো দল আছে:

- 🔍 **বিশ্লেষক দল (Encoder):** তারা মূল ইংরেজি বাক্যটি পড়ে, প্রতিটি শব্দের মানে ও সম্পর্ক বোঝে এবং একটি বিস্তারিত "বোঝাপড়ার রিপোর্ট" তৈরি করে।
- 📝 **লেখক দল (Decoder):** তারা সেই রিপোর্ট দেখে বাংলা অনুবাদ লেখে, একটি একটি করে শব্দ।

Add & Norm হলো সেই **Quality Control** টিম, যারা প্রতিটি ধাপের পরে নিশ্চিত করে যে কাজ সঠিকপথে আছে এবং কেউ পথ হারিয়ে ফেলেনি।

Feed Forward Network হলো প্রতিটি **ব্যক্তিগত বিশেষজ্ঞ**, যিনি নিজের জ্ঞান দিয়ে প্রতিটি শব্দের উপর আলাদাভাবে চিন্তা করেন।

বাস্তব জীবনের উদাহরণ

রান্নার উদাহরণ:

ধরো তুমি একটি বিখ্যাত রেস্টোরাঁয় কাজ করো:

Input Order (উপাদান):

"আমি বিরিয়ানি খেতে চাই"



 Head Chef (Encoder) – উপাদান বিশ্লেষণ

- চাল কোথায়? মাংস কোথায়? মশলা কী লাগবে?
- প্রতিটি উপাদানের relationship বোঝে



 Recipe Card (Encoder Output) – সম্পূর্ণ রেসিপি তৈরি



 Cook (Decoder) – একটু একটু করে রান্না করে

- প্রথমে চাল ধোয়, তারপর মশলা ভাজে...
- Recipe Card দেখে, আগের ধাপ মনে রাখে



 Final Dish (Output): "I want to eat biryani"

মূল সমস্যা যা Transformer সমাধান করে:

- পুরনো RNN/LSTM: শব্দ একটি একটি করে প্রসেস করত, দূরের শব্দ ভুলে যেত
- Transformer: সব শব্দ একসাথে দেখে, যেকোনো দূরত্বের সম্পর্ক ধরতে পারে
- Parallelization: সব শব্দ একসাথে GPU-তে প্রসেস হয় → অনেক দ্রুত

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ সম্পূর্ণ Transformer-এর বড় ছবি

একটি Standard Transformer (Vaswani et al., 2017) এর দুটো প্রধান অংশ:

Encoder Stack: N টি Encoder Block একের পর এক সাজানো (মূল paper-এ N=6)

Decoder Stack: N টি Decoder Block একের পর এক সাজানো (মূল paper-এ N=6)

২.২ Encoder Block — ভেতরের কাজ

প্রতিটি Encoder Block-এ থাকে মাত্র **দুটো Sub-Layer**:

Sub-Layer 1: Multi-Head Self-Attention

- Input sequence-এর প্রতিটি শব্দ, বাকি সব শব্দের দিকে "মনোযোগ" দেয়
- "The bank was steep" — এখানে 'bank' কি নদীর পাড়, নাকি ব্যাংক?
- Self-Attention বাকি শব্দগুলো দেখে সিদ্ধান্ত নেয়

Sub-Layer 2: Position-wise Feed-Forward Network (FFN)

- Attention-এর পরে, প্রতিটি শব্দ আলাদাভাবে দুটো Linear Layer-এর মধ্য দিয়ে যায়
- এটি Non-linearity যোগ করে, শব্দের representation আরও সমৃদ্ধ করে

প্রতিটি Sub-Layer-এর পরে: Add & Norm চালানো হয়

Encoder Block:

Input (x)



Multi-Head Self-Attention

Sublayer(x)

Add: $x + \text{Sublayer}(x)$ ← Residual Connection

Norm: LayerNorm(...) ← Layer Normalization



Feed-Forward Network (FFN)

Sublayer(x)

Add: $x + \text{Sublayer}(x)$ 

Norm: LayerNorm(...)



Output (পরবর্তী Encoder Block-এ যায়)

২.৩ Decoder Block — ভেতরের কাজ

প্রতিটি Decoder Block-এ থাকে তিনটি Sub-Layer:

Sub-Layer 1: Masked Multi-Head Self-Attention

- Decoder নিজের output sequence-এর দিকে মনোযোগ দেয়
- **Masked কেন?** কারণ অনুবাদের সময় Decoder শুধু আগের শব্দগুলো দেখতে পারবে, পরের শব্দ নয়
- উদাহরণ: "আমি" লেখার সময়, পরের "বিরিয়ানি" দেখা যাবে না — এটাই Causal Masking

Sub-Layer 2: Cross-Attention (Encoder-Decoder Attention)

- সবচেয়ে গুরুত্বপূর্ণ লিঙ্ক!

- Query আসে Decoder থেকে: "আমি এখন কীসের উত্তর খুঁজছি?"
- Key ও Value আসে Encoder-এর output থেকে: "মূল ইংরেজি বাক্যে কী ছিল?"
- এভাবে Decoder জানতে পারে কোন Input tokens-এ মনোযোগ দিতে হবে

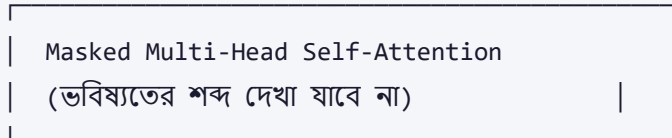
Sub-Layer 3: Feed-Forward Network (FFN)

- Encoder-এর মতোই, প্রতিটি position আলাদাভাবে প্রসেস হয়

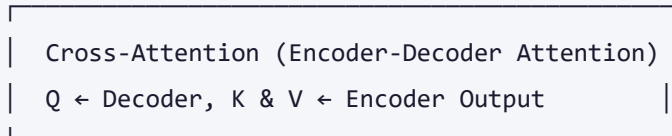
প্রতিটি Sub-Layer-এর পরে: Add & Norm

Decoder Block:

Target Input (ইতিমধ্যে তৈরি হওয়া output)



Add & Norm



Add & Norm



Add & Norm



Output (পরবর্তী Decoder Block-এ যায়)

২.৪ Add & Norm — কেন দরকার?

Residual Connection (Add):

- গভীর নেটওয়ার্কে (6, 12, 24 layer) Vanishing Gradient সমস্যা হয়
- $x \rightarrow \text{Sublayer}(x) \rightarrow x + \text{Sublayer}(x)$ এই shortcut gradient-কে সরাসরি ফ্লো করতে দেয়
- যদি Sublayer কিছু না শেখে, তাহলে x সরাসরি পাস হয়ে যায় $\rightarrow$ নিরাপদ!

Layer Normalization (Norm):

- প্রতিটি শব্দের representation-এর mean=0, std=1 করে দেয়
- Training স্থিতিশীল রাখে, দ্রুত converge করে

২.৫ Feed-Forward Network — বিস্তারিত

FFN হলো একটি ছোট 2-layer MLP, প্রতিটি position-এ আলাদাভাবে প্রয়োগ হয়:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

- $d_{\text{model}} = 512$ (Attention-এর size)
- $d_{\text{ff}} = 2048$ (FFN-এর ভেতরের size — 4 গুণ বড়!)
- প্রথম layer: $512 \rightarrow 2048$ (প্রসারিত)
- দ্বিতীয় layer: $2048 \rightarrow 512$ (সংকুচিত)

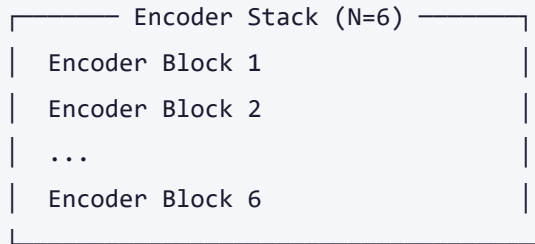
কেন 4x বড়? Attention শব্দের সম্পর্ক ধরে, FFN সেই সম্পর্ক থেকে গভীর অর্থ বের করে।

২.৬ সম্পূর্ণ Data Flow

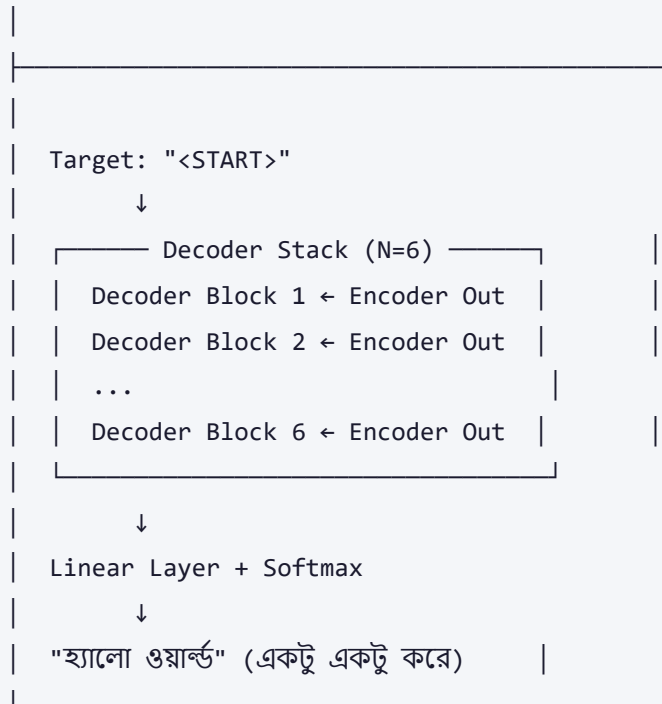
Input Sentence: "Hello World"



Token Embedding + Positional Encoding



↓ (Encoder Output – সব blocks-এর combined wisdom)



৩. Math / Theory

৩.১ Scaled Dot-Product Attention (পূর্ববর্তী নোট থেকে)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q = Query matrix
- K = Key matrix
- V = Value matrix

- d_k = Key dimension (scaling factor)

৩.২ Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

যেখানে প্রতিটি head:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

৩.৩ Add & Norm (Residual + LayerNorm)

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Layer Normalization:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- μ = mean of the layer's activations
- σ^2 = variance
- ϵ = ছোট সংখ্যা (numerical stability-র জন্য, যেমন $1e-6$)
- γ (gamma) এবং β (beta) = learnable parameters

Manual Calculation example:

ধরো একটি layer-এর activation: $x = [2, 4, 6, 8]$

- $\mu = (2+4+6+8)/4 = 5$
- $\sigma^2 = ((2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2)/4 = (9+1+1+9)/4 = 5$
- $\sigma = \sqrt{5} \approx 2.236$

Normalized: $\hat{x} = [(2-5)/2.236, (4-5)/2.236, (6-5)/2.236, (8-5)/2.236]$

$= [-1.342, -0.447, 0.447, 1.342]$

এখন $\gamma=1, \beta=0$ হলে output $= \hat{x}$ (কিন্তু model এই parameters শিখবে)

৩.৪ Position-wise Feed-Forward Network

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

মূল paper-এর dimensions:

- $d_{\text{model}} = 512$
- $d_{\text{ff}} = 2048$
- $W_1: 512 \times 2048$

- $W_2: 2048 \times 512$

Manual Calculation (simplified, $d_{\text{model}}=4$, $d_{\text{ff}}=8$):

$x = [0.5, -0.3, 0.8, -0.1]$ (size: 4)

Step 1: $xW_1 + b_1 \rightarrow$ size: 8 (কম্প্লিট)

Step 2: $\text{ReLU}(\text{previous}) \rightarrow$ negative মান = 0

Step 3: $\text{ReLU_output} \times W_2 + b_2 \rightarrow$ size: 4 (আবার 512-তে ফিরে)

৩.৫ Decoder-এর Causal Masking

Future tokens দেখা ঠেকাতে, একটি mask ব্যবহার হয়:

	Position: 0	1	2	3
Position 0:	[0, -∞, -∞, -∞]			
Position 1:	[0, 0, -∞, -∞]			
Position 2:	[0, 0, 0, -∞]			
Position 3:	[0, 0, 0, 0]			

$-\infty$ মানে Softmax-এর পরে এই position-এর weight = 0 হয়ে যায়।

৩.৬ Final Linear + Softmax

$$P(\text{word}_i) = \text{Softmax}(hW_{\text{vocab}})$$

- h = Decoder-এর final output (size: $d_{\text{model}} = 512$)
- W_{vocab} = Linear matrix (size: $512 \times \text{vocab_size}$)
- vocab_size = সাধারণত 30,000 থেকে 50,000

8. Code Example (Python)

```

import torch
import torch.nn as nn
import math

# =====
# ১. Positional Encoding – শব্দের অবস্থান জানানো
# =====

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        # Dropout layer তৈরি করা
        self.dropout = nn.Dropout(p=dropout)

        # Positional encoding matrix তৈরি করা
        pe = torch.zeros(max_len, d_model) # (max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1).float() # (max_len, 1)

        # Sine ও Cosine frequencies তৈরি
        div_term = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )

        # জোড় index-এ sin, বেজোড় index-এ cos
        pe[:, 0::2] = torch.sin(position * div_term) # even columns
        pe[:, 1::2] = torch.cos(position * div_term) # odd columns

        # (1, max_len, d_model) shape-এ আনা batch support-এর জন্য
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe) # trainable নয়, কিন্তু state-এ থাকবে

    def forward(self, x):
        # x shape: (batch_size, seq_len, d_model)
        # x-এর সাথে positional encoding যোগ করা
        x = x + self.pe[:, :x.size(1), :]
        return self.dropout(x)

# =====
# ২. Multi-Head Attention Layer
# =====

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()

```

```

# d_model অবশ্যই num_heads দিয়ে বিভাজ্য হতে হবে
assert d_model % num_heads == 0, "d_model must be divisible by num_heads"

self.d_model = d_model      # মোট dimension (e.g., 512)
self.num_heads = num_heads  # মাথার সংখ্যা (e.g., 8)
self.d_k = d_model // num_heads # প্রতি মাথার dimension (512/8 = 64)

# Q, K, V এবং Output-এর জন্য Linear layers
self.W_q = nn.Linear(d_model, d_model) # Query projection
self.W_k = nn.Linear(d_model, d_model) # Key projection
self.W_v = nn.Linear(d_model, d_model) # Value projection
self.W_o = nn.Linear(d_model, d_model) # Output projection

def scaled_dot_product_attention(self, Q, K, V, mask=None):
    """Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) * V"""
    # QK^T হিসাব করা
    scores = torch.matmul(Q, K.transpose(-2, -1)) # (... , seq_q, seq_k)
    # sqrt(d_k) দিয়ে scale করা
    scores = scores / math.sqrt(self.d_k)

    # Mask প্রয়োগ করা (Decoder-এর জন্য future masking)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    # Softmax দিয়ে attention weights
    attention_weights = torch.softmax(scores, dim=-1)

    # Value-এর সাথে গুণ করে output বের করা
    output = torch.matmul(attention_weights, V)
    return output, attention_weights

def split_heads(self, x, batch_size):
    """Single tensor-কে num_heads ভাগে split করা"""
    # (batch, seq, d_model) → (batch, seq, num_heads, d_k)
    x = x.view(batch_size, -1, self.num_heads, self.d_k)
    # (batch, num_heads, seq, d_k)
    return x.transpose(1, 2)

def forward(self, query, key, value, mask=None):
    batch_size = query.size(0)

    # Linear projection করা
    Q = self.W_q(query) # (batch, seq_q, d_model)

```

```

K = self.W_k(key)    # (batch, seq_k, d_model)
V = self.W_v(value)  # (batch, seq_v, d_model)

# Multiple heads-এ split করা
Q = self.split_heads(Q, batch_size) # (batch, heads, seq_q, d_k)
K = self.split_heads(K, batch_size) # (batch, heads, seq_k, d_k)
V = self.split_heads(V, batch_size) # (batch, heads, seq_v, d_k)

# Attention হিসাব করা
attn_output, attn_weights = self.scaled_dot_product_attention(Q, K, V, mask)
# attn_output: (batch, heads, seq_q, d_k)

# Heads-গুলো আবার একত্রিত করা
# (batch, heads, seq, d_k) → (batch, seq, heads, d_k)
attn_output = attn_output.transpose(1, 2).contiguous()
# (batch, seq, d_model)
attn_output = attn_output.view(batch_size, -1, self.d_model)

# Final linear projection
output = self.W_o(attn_output) # (batch, seq, d_model)
return output

```

```

# =====
# ৩. Feed-Forward Network
# =====

```

```

class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        # প্রথম linear layer: সম্প্রসারণ (512 → 2048)
        self.linear1 = nn.Linear(d_model, d_ff)
        # দ্বিতীয় linear layer: সংকোচন (2048 → 512)
        self.linear2 = nn.Linear(d_ff, d_model)
        # Activation function – ReLU দিয়ে non-linearity
        self.relu = nn.ReLU()
        # Dropout – overfitting রোধ করতে
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # FFN(x) = max(0, xW1 + b1)W2 + b2
        x = self.linear1(x) # (batch, seq, d_ff)
        x = self.relu(x)    # Negative মান শূন্য হয়ে যায়
        x = self.dropout(x) # Training-এ কিছু মান বাদ দেওয়া
        x = self.linear2(x) # (batch, seq, d_model)

```

```

        return x

# =====
# 8. Encoder Block – দুটো Sub-layer
# =====

class EncoderBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        # Sub-Layer 1: Multi-Head Self-Attention
        self.self_attention = MultiHeadAttention(d_model, num_heads)
        # Sub-Layer 2: Feed-Forward Network
        self.feed_forward = FeedForwardNetwork(d_model, d_ff, dropout)

        # Layer Normalization – প্রতিটি sub-layer-এর পরে
        self.norm1 = nn.LayerNorm(d_model) # Attention-এর পরে
        self.norm2 = nn.LayerNorm(d_model) # FFN-এর পরে

        # Dropout
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, src_mask=None):
        # — Sub-Layer 1: Self-Attention + Add & Norm —
        # Self-attention: Q=K=V=x (নিজের দিকে মনোযোগ)
        attn_output = self.self_attention(x, x, x, src_mask)
        # Residual connection + Layer Norm
        x = self.norm1(x + self.dropout(attn_output)) # Add & Norm

        # — Sub-Layer 2: FFN + Add & Norm —
        ffn_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ffn_output)) # Add & Norm

        return x # পরবর্তী Encoder Block-এ যাবে

# =====
# ৫. Decoder Block – তিনটি Sub-layer
# =====

class DecoderBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        # Sub-Layer 1: Masked Self-Attention (নিজের output দেখে)
        self.masked_self_attention = MultiHeadAttention(d_model, num_heads)
        # Sub-Layer 2: Cross-Attention (Encoder-এর output দেখে)
        self.cross_attention = MultiHeadAttention(d_model, num_heads)

```

```

# Sub-Layer 3: Feed-Forward Network
self.feed_forward = FeedForwardNetwork(d_model, d_ff, dropout)

# তিনটি Layer Normalization
self.norm1 = nn.LayerNorm(d_model) # Masked Self-Attention-এর পরে
self.norm2 = nn.LayerNorm(d_model) # Cross-Attention-এর পরে
self.norm3 = nn.LayerNorm(d_model) # FFN-এর পরে

self.dropout = nn.Dropout(dropout)

def forward(self, x, encoder_output, src_mask=None, tgt_mask=None):
    # — Sub-Layer 1: Masked Self-Attention —
    # tgt_mask: ভবিষ্যতের শব্দ দেখা ঠেকানো
    attn1 = self.masked_self_attention(x, x, x, tgt_mask)
    x = self.norm1(x + self.dropout(attn1)) # Add & Norm

    # — Sub-Layer 2: Cross-Attention —
    # Q = Decoder থেকে (x)
    # K, V = Encoder-এর output থেকে
    attn2 = self.cross_attention(x, encoder_output, encoder_output, src_mask)
    x = self.norm2(x + self.dropout(attn2)) # Add & Norm

    # — Sub-Layer 3: FFN —
    ffn_out = self.feed_forward(x)
    x = self.norm3(x + self.dropout(ffn_out)) # Add & Norm

    return x # পরবর্তী Decoder Block-এ যাবে

# =====
# ৬. সম্পূর্ণ Transformer Model
# =====
class Transformer(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model=512,
                  num_heads=8, num_encoder_layers=6, num_decoder_layers=6,
                  d_ff=2048, max_seq_len=5000, dropout=0.1):
        super().__init__()

        # — Embeddings —
        # Source (input) শব্দের embedding
        self.src_embedding = nn.Embedding(src_vocab_size, d_model)
        # Target (output) শব্দের embedding
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)

```

```

# — Positional Encoding —
self.positional_encoding = PositionalEncoding(d_model, max_seq_len, dropout)

# — Encoder Stack: N টি Encoder Block —
self.encoder_layers = nn.ModuleList([
    EncoderBlock(d_model, num_heads, d_ff, dropout)
    for _ in range(num_encoder_layers)
])

# — Decoder Stack: N টি Decoder Block —
self.decoder_layers = nn.ModuleList([
    DecoderBlock(d_model, num_heads, d_ff, dropout)
    for _ in range(num_decoder_layers)
])

# — Final Linear Projection → Vocabulary —
# d_model → tgt_vocab_size
self.output_projection = nn.Linear(d_model, tgt_vocab_size)

# — Layer Normalization (Encoder ও Decoder-এর শেষে) —
self.encoder_norm = nn.LayerNorm(d_model)
self.decoder_norm = nn.LayerNorm(d_model)

def encode(self, src, src_mask=None):
    """শুধু Encoder চালানো"""
    # Token → Embedding → Positional Encoding
    x = self.src_embedding(src) * math.sqrt(self.src_embedding.embedding_dim)
    x = self.positional_encoding(x)

    # N টি Encoder Block একে একে
    for encoder_layer in self.encoder_layers:
        x = encoder_layer(x, src_mask)

    return self.encoder_norm(x) # Final normalized output

def decode(self, tgt, encoder_output, src_mask=None, tgt_mask=None):
    """শুধু Decoder চালানো"""
    # Target Token → Embedding → Positional Encoding
    x = self.tgt_embedding(tgt) * math.sqrt(self.tgt_embedding.embedding_dim)
    x = self.positional_encoding(x)

    # N টি Decoder Block একে একে
    for decoder_layer in self.decoder_layers:

```

```

        x = decoder_layer(x, encoder_output, src_mask, tgt_mask)

    return self.decoder_norm(x) # Final normalized output

def forward(self, src, tgt, src_mask=None, tgt_mask=None):
    """সম্পূর্ণ Forward Pass"""
    # ১. Encoder দিয়ে input প্রসেস
    encoder_output = self.encode(src, src_mask)

    # ২. Decoder দিয়ে output তৈরি (Encoder output ব্যবহার করে)
    decoder_output = self.decode(tgt, encoder_output, src_mask, tgt_mask)

    # ৩. Final projection → Vocabulary probabilities
    output = self.output_projection(decoder_output)
    return output

# =====
# ৭. Helper Function: Causal Mask তৈরি করা
# =====
def create_causal_mask(seq_len):
    """ভবিষ্যতের টোকেন দেখা ঠেকাতে Lower Triangular Mask"""
    # torch.tril: lower triangular matrix
    mask = torch.tril(torch.ones(seq_len, seq_len))
    return mask.unsqueeze(0).unsqueeze(0) # (1, 1, seq_len, seq_len)

# =====
# ৮. Test করা – একটি ছোট উদাহরণ
# =====
if __name__ == "__main__":
    # Hyperparameters (ছোট করে test করা)
    SRC_VOCAB = 1000 # Input vocabulary size
    TGT_VOCAB = 1000 # Output vocabulary size
    D_MODEL = 128 # Model dimension (মূল paper-এ 512)
    NUM_HEADS = 4 # Attention heads (মূল paper-এ 8)
    NUM_LAYERS = 2 # Encoder/Decoder layers (মূল paper-এ 6)
    D_FF = 256 # FFN intermediate size (মূল paper-এ 2048)
    BATCH_SIZE = 2 # একসাথে কয়টি sentence
    SRC_SEQ_LEN = 10 # Input sequence length
    TGT_SEQ_LEN = 8 # Target sequence length

    print("=" * 60)
    print("🚀 সম্পূর্ণ Transformer Architecture – Test")
    print("=" * 60)

```



```

# Model তৈরি করা
model = Transformer(
    src_vocab_size=SRC_VOCAB,
    tgt_vocab_size=TGT_VOCAB,
    d_model=D_MODEL,
    num_heads=NUM_HEADS,
    num_encoder_layers=NUM_LAYERS,
    num_decoder_layers=NUM_LAYERS,
    d_ff=D_FF
)

# Model parameter count
total_params = sum(p.numel() for p in model.parameters())
print(f"\n🌈 Model Parameters: {total_params:,}")
print(f"   D_Model: {D_MODEL}, Heads: {NUM_HEADS}, Layers: {NUM_LAYERS}")

# Fake input data তৈরি করা (random token IDs)
src = torch.randint(0, SRC_VOCAB, (BATCH_SIZE, SRC_SEQ_LEN))
tgt = torch.randint(0, TGT_VOCAB, (BATCH_SIZE, TGT_SEQ_LEN))

print(f"\n📦 Input shapes:")
print(f"   Source: {src.shape} (batch={BATCH_SIZE}, src_len={SRC_SEQ_LEN})")
print(f"   Target: {tgt.shape} (batch={BATCH_SIZE}, tgt_len={TGT_SEQ_LEN})")

# Causal mask তৈরি করা
tgt_mask = create_causal_mask(TGT_SEQ_LEN)
print(f"\n🐦 Causal Mask shape: {tgt_mask.shape}")
print("   (Lower triangular – ভবিষ্যতের শব্দ দেখা যাবে না)")
print(f"\nCausal Mask (4x4 example):\n{create_causal_mask(4).squeeze()}")

# Forward pass
print("\n⚡ Forward pass চালানো হচ্ছে...")
model.eval() # Evaluation mode
with torch.no_grad(): # Gradient calculation বন্ধ
    output = model(src, tgt, tgt_mask=tgt_mask)

print(f"\n📦 Output shape: {output.shape}")
print(f"   (batch={BATCH_SIZE}, tgt_len={TGT_SEQ_LEN}, vocab={TGT_VOCAB})")
print("\n✅ প্রতিটি position-এ vocab_size সংখ্যক probability score আছে!")

# Softmax দিয়ে probabilities বের করা
probs = torch.softmax(output, dim=-1)

```

```
predicted_tokens = torch.argmax(probs, dim=-1)
print(f"\n 🎯 Predicted Token IDs:\n {predicted_tokens}")

# Encoder-only test
print("\n" + "=" * 60)
print(" 🔍 Encoder-only Test (BERT-style models-এর মতো)")
print("=" * 60)
encoder_out = model.encode(src)
print(f"Encoder Output shape: {encoder_out.shape}")
print(f" (batch={BATCH_SIZE}, src_len={SRC_SEQ_LEN}, d_model={D_MODEL})")
print("\n ✅ Test সফল! Transformer সঠিকভাবে কাজ করছে।")
```

Expected Output:

=====

🚗 সম্পূর্ণ Transformer Architecture – Test

=====

📊 Model Parameters: 1,578,752

D_Model: 128, Heads: 4, Layers: 2

📏 Input shapes:

Source: torch.Size([2, 10]) (batch=2, src_len=10)

Target: torch.Size([2, 8]) (batch=2, tgt_len=8)

👉 Causal Mask shape: torch.Size([1, 1, 8, 8])

(Lower triangular – ভবিষ্যতের শব্দ দেখা যাবে না)

Causal Mask (4x4 example):

```
tensor([[1., 0., 0., 0.],
        [1., 1., 0., 0.],
        [1., 1., 1., 0.],
        [1., 1., 1., 1.]])
```

⚡ Forward pass চালানো হচ্ছে...

📏 Output shape: torch.Size([2, 8, 1000])

(batch=2, tgt_len=8, vocab=1000)

✅ প্রতিটি position-এ vocab_size সংখ্যক probability score আছে!

🌟 Predicted Token IDs:

```
tensor([[..., ..., ...], [..., ..., ...]])
```

=====

🔍 Encoder-only Test (BERT-style models-এর মতো)

=====

Encoder Output shape: torch.Size([2, 10, 128])

(batch=2, src_len=10, d_model=128)

✅ Test সফল! Transformer সঠিকভাবে কাজ করছে।

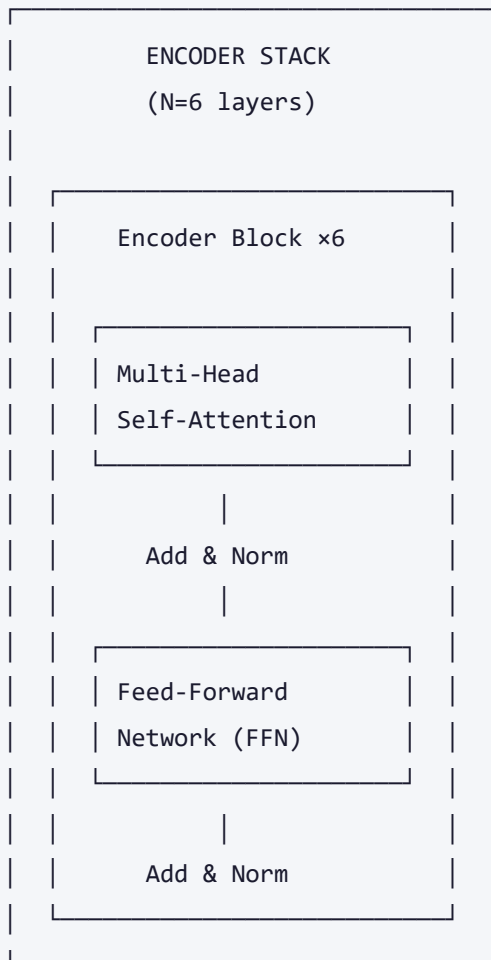
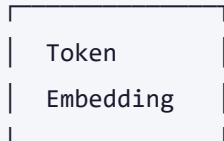
৫. 🎨 Visual / Diagram

৫.১ সম্পূর্ণ Transformer Architecture

THE TRANSFORMER (Vaswani et al., 2017)

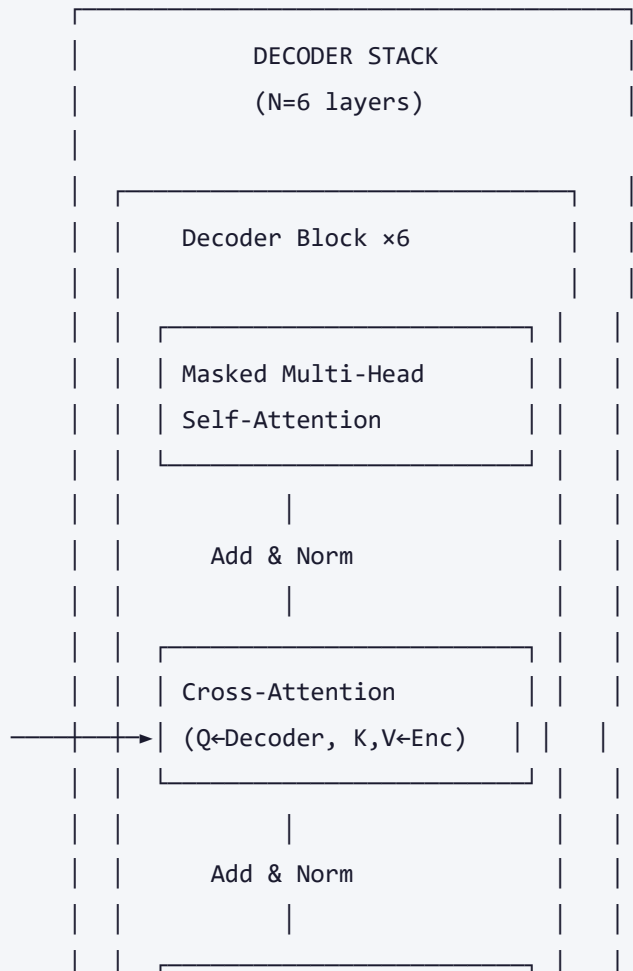
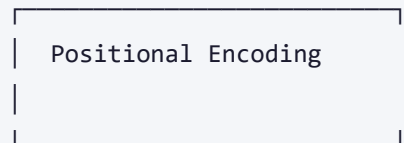
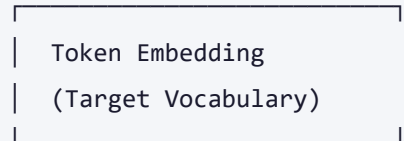
INPUT SEQUENCE

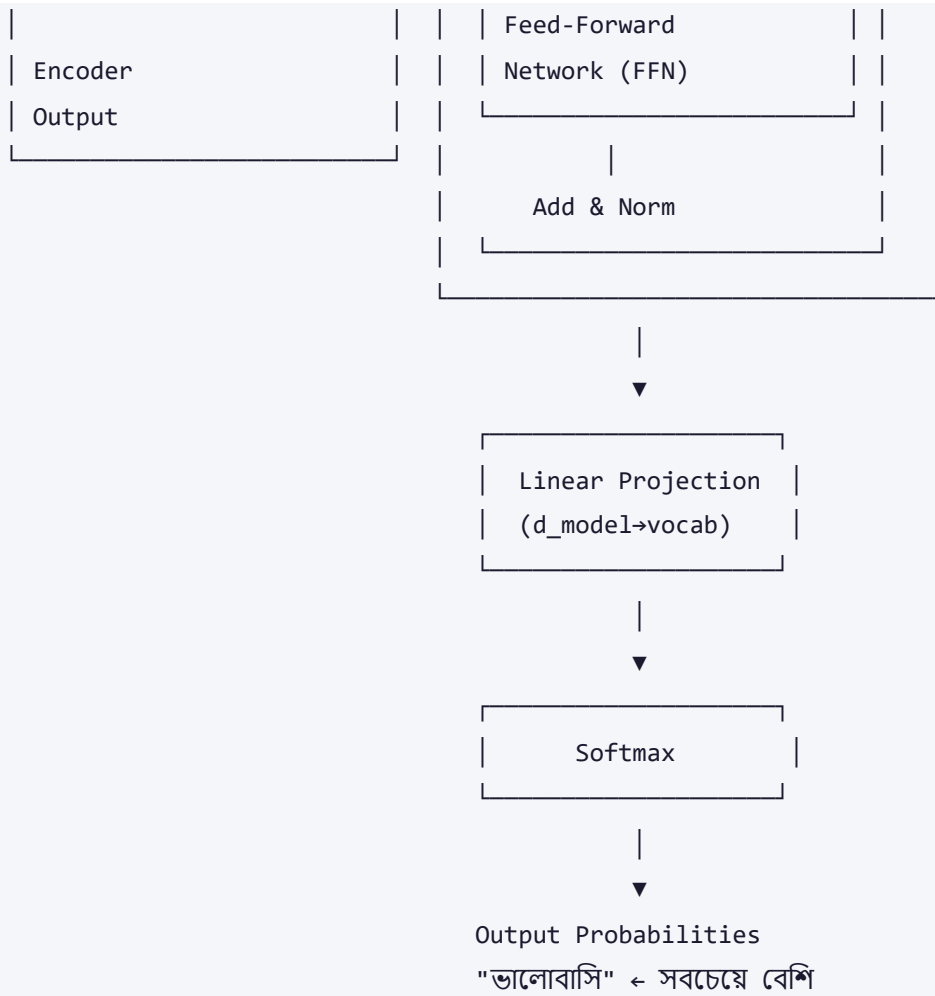
"I love you"



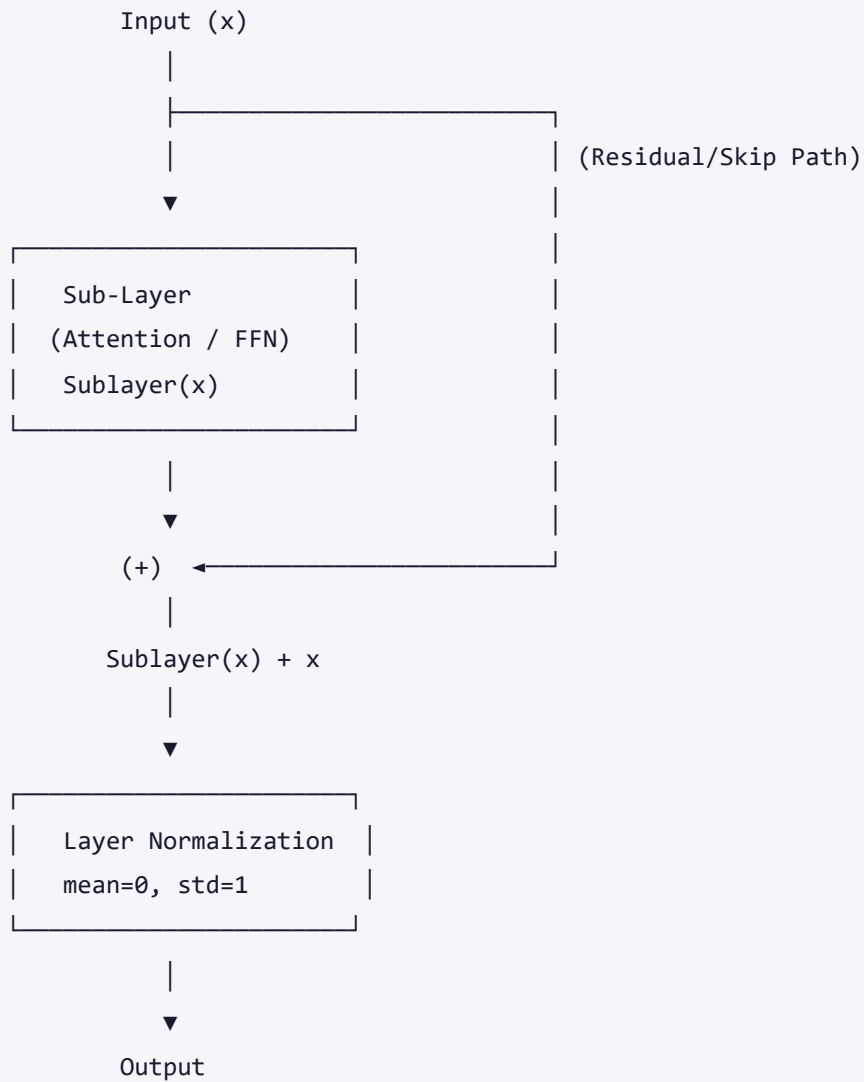
OUTPUT SEQUENCE (Shifted Right)

"<START> আমি তোমাকে"

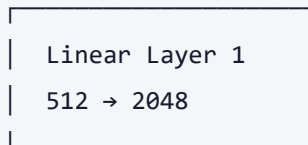
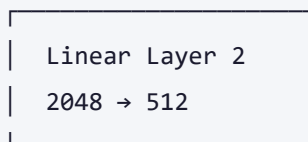




৫.২ Add & Norm (Residual Connection) Diagram



🔗🔄 Feed-Forward Network (FFN) Diagram

Input: d_{model} (512) $W_1: (512 \times 2048)$  $\max(0, x) \rightarrow$ Negative মান বাদ $W_2: (2048 \times 512)$ Output: d_{model} (512)

৫.8 Causal Masking Visualization

Decoder Attention Matrix (4 tokens: "আমি ভালো তোমাকে বাসি")

	আমি	ভালো	তোমাকে	বাসি
আমি	[   ]			
ভালো	[   ]			
তোমাকে	[   ]			
বাসি	[   ]			

✓ = দেখতে পারে (গণনায় অংশ নেয়)

✗ = দেখতে পারে না ($\text{mask} = -\infty \rightarrow \text{softmax} \approx 0$)

৬. Real-world Use Cases

Use Case ১: Machine Translation (Google Translate)

- **কোম্পানি:** Google
- **কীভাবে:** Encoder বাংলা বাক্য বোঝে, Decoder ইংরেজি তৈরি করে
- **Model:** Google-এর Neural Machine Translation System (based on Transformer)
- **Impact:** প্রতিদিন 100+ ভাষায় কোটি কোটি অনুবাদ

Use Case ২: ChatGPT / GPT-4

- **কোম্পানি:** OpenAI
- **কীভাবে:** Decoder-only Transformer (GPT ধরনের)
- **বিশেষত্ব:** Causal Masking ব্যবহার করে পরবর্তী শব্দ predict করে
- **Scale:** GPT-4 এ ট্রিলিয়ন+ parameters, হাজারো Decoder blocks

Use Case ৩: BERT (Bidirectional Text Understanding)

- **কোম্পানি:** Google
- **কীভাবে:** Encoder-only Transformer (BERT ধরনের)
- **বিশেষত্ব:** উভয় দিক থেকে context বোঝে, classification/NER-এ ব্যবহার
- **Application:** Google Search-এর ranking algorithm

Use Case ৪: Whisper (Speech Recognition)

- **কোম্পানি:** OpenAI
- **কীভাবে:** Audio Encoder + Text Decoder Transformer
- **Application:** Speech-to-text, automatic subtitles
- **Impact:** 99টি ভাষায় speech recognition

Use Case ৫: AlphaFold 2 (Protein Structure Prediction)

- **কোম্পানি:** DeepMind (Google)
- **কীভাবে:** Transformer-based architecture amino acid sequence বিশ্লেষণ করে
- **Impact:** জীববিজ্ঞানে যুগান্তকারী আবিষ্কার — 2021-এ বিজ্ঞানের সেরা ঘটনা

৭. ⚖️ Pros & Cons

সুবিধা ✅

Parallelization: RNN-এর মতো sequential নয়, সব token একসাথে প্রসেস হয়

Long-range Dependencies: দূরের শব্দের সম্পর্কও perfectly ধরতে পারে

Scalability: বড় model ও বেশি data → better performance

Transfer Learning: Pre-train করে fine-tune করা যায় (BERT, GPT)

Flexibility: Encoder-only, Decoder-only, বা দুটোই ব্যবহার করা যায়

SOTA Performance: NLP, Vision, Audio — সব task-এ state-of-the-art

অসুবিধা ❌

Memory Usage: $O(n^2)$ attention matrix — দীর্ঘ sequence-এ RAM বেশি লাগে

Positional Encoding প্রয়োজন: Sequence order নিজে বোঝে না, আলাদা encode করতে হয়

Computationally Expensive: ছোট dataset-এ RNN/LSTM-এর চেয়ে বেশি resource নেয়

Interpretability: কেন নির্দিষ্ট output দিল, বোঝা কঠিন (Black box)

Data Hungry: ভালো performance-এর জন্য প্রচুর training data দরকার

Quadratic Complexity: Sequence length n হলে $O(n^2)$ complexity

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: Masking ভুলে যাওয়া

❌ ভুল – Decoder-এ mask না দিলে future leakage হয়
`output = decoder(tgt, encoder_out)` # mask নেই!

✅ সঠিক – Causal mask অবশ্যই দিতে হবে
`tgt_mask = create_causal_mask(tgt_seq_len)`
`output = decoder(tgt, encoder_out, tgt_mask=tgt_mask)`

ভুল ২: Residual Connection-এর আগে dimension mismatch

❌ ভুল – Input ও Sublayer output-এর dimension আলাদা হলে Add করা যাবে না
 # Input: (batch, seq, 512) + Sublayer output: (batch, seq, 256) → Error!

✅ সঠিক – সব layer-এ d_{model} একই রাখতে হবে

ভুল ৩: Layer Norm-এর অবস্থান

- মূল paper: Add তারপর Norm (Post-LN)
- আধুনিক practice: Norm তারপর Sublayer (Pre-LN) — বেশি stable

Post-LN (মূল paper): $x = \text{LayerNorm}(x + \text{Sublayer}(x))$
 # Pre-LN (আধুনিক): $x = x + \text{Sublayer}(\text{LayerNorm}(x))$

ভুল ৪: Embedding Scaling ভুলে যাওয়া

✅ সঠিক – embedding-কে $\sqrt{d_{model}}$ দিয়ে scale করতে হয়
 $x = \text{self.embedding}(\text{tokens}) * \text{math.sqrt}(\text{self.d_model})$
 # কারণ: positional encoding-এর size-এর সাথে match করতে হয়

ভুল ৫: Training-এ Teacher Forcing না বোঝা

- Training-এ Decoder সত্যিকারের target token দেখে (Teacher Forcing)
- Inference-এ নিজের আগের output দেখে (Autoregressive)
- এই দুটো পার্থক্য না বুঝলে implementation ভুল হবে

ভুল ৬: Cross-Attention-এর Q, K, V বিভ্রান্তি

❌ ভুল
 $\text{cross_attn} = \text{attention}(\text{encoder_out}, \text{encoder_out}, \text{decoder_out})$ # Q ভুল!

✅ সঠিক – Q=Decoder, K=V=Encoder
 $\text{cross_attn} = \text{attention}(\text{decoder_out}, \text{encoder_out}, \text{encoder_out})$

৯. Related Topics

আগে জানা দরকার (Prerequisites):

1. **Self-Attention & Q, K, V** — Transformer-এর হৃদয়
2. **Multi-Head Attention & Positional Encoding** — সম্পূর্ণ attention mechanism
3. **Seq2Seq ও Attention Mechanism** — Transformer-এর ইতিহাস
4. **Backpropagation ও Gradient Descent** — কীভাবে model শেখে
5. **Layer Normalization vs Batch Normalization** — পার্থক্য বোঝা

পরে কী শেখা উচিত:

1. **BERT (Encoder-only)** — Text classification, NER, Question Answering
2. **GPT ধরনের Models (Decoder-only)** — Language generation, ChatGPT
3. **T5/BART (Encoder-Decoder)** — Translation, Summarization
4. **Efficient Transformers** — Longformer, Linformer ($O(n^2)$ সমস্যার সমাধান)
5. **Vision Transformer (ViT)** — Image recognition-এ Transformer
6. **Fine-tuning & LORA** — Pre-trained model কাস্টমাইজ করা

১০. Memory Tricks

মনে রাখার কৌশল:

"E-3, D-3 মনে রাখো":

- **Encoder:** ২টি Sub-layer (Self-Attention + FFN)
- **Decoder:** ৩টি Sub-layer (Masked Self-Att + Cross-Att + FFN)
- প্রতিটির পরে: **Add & Norm**

Add & Norm মনে রাখার Trick:

"আগে যোগ করো (Residual), তারপর সাজাও (Normalize)"

"ক্লাস শেষে নতুন প্রশ্নের সাথে আগের জ্ঞান যোগ করো, তারপর সব *normalize* করো"

FFN মনে রাখার Trick:

"প্রথমে **বড়** করো ($512 \rightarrow 2048$), তারপর **ছোট** করো ($2048 \rightarrow 512$)"

"এটা আগে বিস্তার করে গভীরে ভাবে, তারপর সংক্ষিপ্ত করে"

Cross-Attention Trick:

"*Q*uey আসে *D*ecoder থেকে, *K*ey ও *V*alue আসে *E*ncoder থেকে"

"মনে রেখো: $Q=D$, $K=E$, $V=E$ অথবা Question from Decoder, Knowledge from Encoder"

Causal Mask:

"নিজের ভবিষ্যত দেখা যাবে না — Lower Triangular Matrix"

১ লাইনে সারসংক্ষেপ:

"*Transformer* = *Encoder* (পড়ো ও বোঝো) + *Decoder* (লেখো) + *Add & Norm* (স্থিতিশীল রাখো) + *FFN* (গভীরে ভাবো) — এই চারের সমন্বয়ে আধুনিক AI!"

 সম্পদ ও রেফারেন্স

সম্পদ	লিঙ্ক	কেন দেখবে?
Original Paper	"Attention Is All You Need" (Vaswani et al., 2017)	Transformer-এর মূল source
The Annotated Transformer	Harvard NLP Group	কোড সহ বিস্তারিত ব্যাখ্যা
Illustrated Transformer	Jay Alammar Blog	সেরা visual explanation
d2l.ai	dive into deep learning	Interactive implementation

 তৈরির তারিখ: ২০২৬-০৪-১১ |  সিরিজ: Transformers & LLMs — পর্ব ৫/৫

 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes